King Mongkut's University of Technology Thonburi
Faculty of Engineering, Department of Computer Engineering
**CPE 333 Operating Systems, 1/2024**

**PROBLEM SESSION Week #3: Process Manipulation and Monitoring.**

---

# Static Storage Class

**Step 1:** Compile and run this program 3 times, copy the output and put it in your report, explain why the result show up that way

```
The value of y is 6
The address of y is 0x5595aa281010

The value of y is 7
The address of y is 0x5595aa281010

The value of y is 8
The address of y is 0x5595aa281010

The value of y is 6
The address of y is 0x62b76f30f010

The value of y is 7
The address of y is 0x62b76f30f010

The value of y is 8
The address of y is 0x62b76f30f010

The value of y is 6
The address of y is 0x56a375d25010

The value of y is 7
The address of y is 0x56a375d25010

The value of y is 8
The address of y is 0x56a375d25010
```

**Explanation**

`auto` gives x automatic storage duration localy. The variable is initialized to 3 when `main()` begins. During each loop iteration, x is decremented by 1. Its values are 3, 2, and 1 during the three iterations, after which it becomes 0 and the condition `x > 0` becomes false. Therefore, the loop executes exactly three times.

The variable y is declared with static. This means that y is created only once and exists for the entire lifetime of the program. That is why y went from 6 to 7 and 8 because of y++ in each loop. However, the address may be different when the program is executed again because the program's memory layout can change between executions when PIE and ASLR are enabled.

Address Space Layout Randomization (ASLR): ASLR randomizes the memory addresses of different parts of a program each time it starts.

Position-Independent Executable (PIE) allows the main executable, including its code and static data, to be loaded at different virtual memory addresses.

PSLR can randomize the addresses of the stack, heap, and, when PIE is enabled, the main executable address is randomize.

**Step 2:** Modify the code by delete the `static` word, compile, and re-run the program.
**Step 3:** Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

```
The value of y is 6
The address of y is 0x7ffecc8e85c0

The value of y is 6
The address of y is 0x7ffecc8e85c0

The value of y is 6
The address of y is 0x7ffecc8e85c0

The value of y is 6
The address of y is 0x7ffc16fe2260

The value of y is 6
The address of y is 0x7ffc16fe2260

The value of y is 6
The address of y is 0x7ffc16fe2260

The value of y is 6
The address of y is 0x7fff2e816410

The value of y is 6
The address of y is 0x7fff2e816410

The value of y is 6
The address of y is 0x7fff2e816410
```

**Explanation**

Without `static`, y becomes an automatic local variable. Its lifetime lasts only until the function/block finishes executing, after which it is destroyed. It is typically stored on the stack. Therefore, their memory addresses can be far apart because they are stored in different memory segments. Each time the loop enters the block, y is initialized to 5. The statement y++ then changes it to 6. When the iteration ends, the lifetime of that automatic variable ends. On the next iteration, y is initialized to 5 again, so the printed value is always 6.

Although a new automatic y is created on each iteration, the compiler can reuse the same stack memory location because the previous y has reached the end of its lifetime. Therefore, in this experiment the address can remain the same across iterations. However, the stack address can change between separate executions because of ASLR.

**Step 4:** Redo step 1-3, but this time compile the program using:

```
“gcc -no-pie -o <your executable object name> <your c code name>”
```

Are they different? Why or why not

**Step 4.1:** With `static` .

```
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018
```

**Explanation**

With static, y keeps its value between loop iterations, so the output is 6, 7, and 8. Its address remains the same because all iterations use the same variable similar to step 1.

When using `-no-pie` , PIE is disabled. This means the executable and statiac data is loaded at a fixed memory address instead of being relocated to a different address. Since the `static` variable y, y has

the address `0x404018` in every execution.

**Step 4.2:** Without `static` .

```
The value of y is 6
The address of y is 0x7ffc878a8570

The value of y is 6
The address of y is 0x7ffc878a8570

The value of y is 6
The address of y is 0x7ffc878a8570

The value of y is 6
The address of y is 0x7ffe606424b0

The value of y is 6
The address of y is 0x7ffe606424b0

The value of y is 6
The address of y is 0x7ffe606424b0

The value of y is 6
The address of y is 0x7ffe8940b570

The value of y is 6
The address of y is 0x7ffe8940b570

The value of y is 6
The address of y is 0x7ffe8940b570
```

### Explanation

Without `static` , y is an automatic variable that is recreated and initialized to 5 during each loop iteration. Therefore, its value is always 6 after `y++` . Its address remains also remain the same within each execution because the same stack location can be reused for y on each loop iteration similar to step 3.

However diffrent from step 4.1, the address changes between separate executions because y is an automatic variable stored on the stack which is outside the range affected by PIE randomization, and

`-no-pie` does not disable ASLR. Therefore, the stack can be located at a different virtual address each time the program runs.

Therefore, `-no-pie` makes the static variable's address fixed in this environment, but it does not affect the stack address of the automatic variable. As a result, the non-static variable shows essentially the same address behavior with or without `-no-pie`.

**Comparison**

| Case | Value of `y` | Address within execution | Address between executions |
|---|---|---|---|
| `static`, normal compilation | 6 -> 7 -> 8 | Same | Change |
| `auto`, normal compilation | 6 -> 6 -> 6 | Same in this experiment | Change |
| `static`, `-no-pie` | 6 -> 7 -> 8 | Same | Same in this environment |
| `auto`, `-no-pie` | 6 -> 6 -> 6 | Same in this experiment | Change |

Overall, the main difference is the effect of `static` and `-no-pie` on the variable's lifetime and address. With static, y retains its value between loop iterations, producing 6 → 7 → 8. Without static, y is recreated and initialized to 5 on each iteration, so the output remains 6 → 6 → 6.

Without `-no-pie`, the address of the static variable changes between separate executions because PIE allows the executable to be relocated and ASLR randomizes the memory layout. When `-no-pie` is used, the main executable is not position-independent, so the static variable's address remains fixed in this environment at 0x404018.

The address of the non-static variable remains the same within an execution because the compiler can reuse the same stack location. However, its address changes between executions because it is stored on the stack, and ASLR can randomize the stack location. Therefore, using `-no-pie` does not make the address of the automatic variable fixed.

# Extern Storage Class

**Step 1:** Compile and run this program 3 times, copy the output and put it in your report, explain why the result show up that way

```
Print value of x: 20
Adress of x (in main function): 0x6096408c2010

Display value of x: 20
Adress of x (in display function): 0x6096408c2010

Print value of x: 20
Adress of x (in main function): 0x5e1a18ca0010

Display value of x: 20
Adress of x (in display function): 0x5e1a18ca0010

Print value of x: 20
Adress of x (in main function): 0x5cc33dfb4010

Display value of x: 20
Adress of x (in display function): 0x5cc33dfb4010
```

**Explanation**

The variable x is a global variable that is initialized to 20. The `extern` keyword allows the `display()` function to access the same global variable x that is defined elsewhere in the program.Therefore, when x is printed in `main()`, its value is 20. When x is printed again inside `display()`, the value is also 20 with the same adress as in `main()` because both functions are accessing the same variable.

However, the address changes between separate executions. This happens similar to `1. Static Storage Class` due to PIE and ASLR still enable.

**Step 2:** Modify the code by delete the `extern` word, compile, and re-run the program.
**Step 3:** Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

```
Print value of x: 0
Adress of x (in main function): 0x7fff82c07e54

Display value of x: 0
Adress of x (in display function): 0x7fff82c07e34

Print value of x: 0
Adress of x (in main function): 0x7ffe0b6ab584

Display value of x: 0
Adress of x (in display function): 0x7ffe0b6ab564

Print value of x: 0
Adress of x (in main function): 0x7fff1070bd64

Display value of x: 0
Adress of x (in display function): 0x7fff1070bd44
```

**Explanation**

Without extern, the x variables used by `main()` and `display()` are separate local variables. Therefore, they have different memory addresses. A newly create variable localy in its own function value is also undefined.

**Step 4:** Redo step 1-3, but this time compile the program using:

“gcc -no-pie -o <your executable object name> <your c code name>”

Are they different? Why or why not

**Step 4.1:** With `extern` .

```
Print value of x: 20
Adress of x (in main function): 0x404018

Display value of x: 20
Adress of x (in display function): 0x404018

Print value of x: 20
Adress of x (in main function): 0x404018

Display value of x: 20
Adress of x (in display function): 0x404018

Print value of x: 20
Adress of x (in main function): 0x404018

Display value of x: 20
Adress of x (in display function): 0x404018
```

**Explanation**

Same with in step 1 both fuction have access to x with `extern` .

The program was compiled using -no-pie. This means the executable is loaded at a fixed memory address instead of being relocated to a different address between executions. Since the `static` variable y is stored in the executable's data section, its address is also fixed. Therefore, y has the address `0x404018` in every execution, exacly the same as in 1.Static Storage Class step 4.1.

**Step 4.2:** Without `extern` .

```
Print value of x: 0
Adress of x (in main function): 0x7fffc1a84ba4

Display value of x: 0
Adress of x (in display function): 0x7fffc1a84b84

Print value of x: 0
Adress of x (in main function): 0x7ffcb85d5c74

Display value of x: 0
Adress of x (in display function): 0x7ffcb85d5c54

Print value of x: 0
Adress of x (in main function): 0x7ffcfbecac14

Display value of x: 0
Adress of x (in display function): 0x7ffcfbecabf4
```

**Explanation**

Same with in step 3 that both fuction have its on x local variable.

Using -no-pie does not disable ASLR. It only disables PIE for the main executable. Therefore, unlike the global variable in the extern case, these local variables still have different and randomized stack addresses between executions.

**Comparison**

| Case | Value of x | Address of x in main function and display fuction | Between executions |
|---|---|---|---|
| `extern` , normal compilation | 20 | Same | Change |
| `auto` , normal compilation | undefined | Diffrent | Change |
| `extern` , `-no-pie` | 20 | Same | Same in this environment |

| Case | Value of x | Address of x in main function and display fuction | Between executions |
| --- | --- | --- | --- |
| `auto` , `-no-pie` | undefined | Diffrent | Change |

## Comparison

The main difference is that `extern` allows `main()` and `display()` to access the same global variable. Therefore, both functions print the same value, `20` , and the address of x is identical in both functions. Without `extern` , the x variables in `main()` and `display()` are separate local variables. Therefore, they have different addresses. If these local variables are not initialized, their values are undefined.

With normal compilation, the address of the global x changes between executions because PIE and ASLR can change the location of the executable.

Same with in `1.` , when compiling with `-no-pie` , PIE is disabled. In this experiment, the global variable x therefore has the fixed address `0x404018` in every execution. However, `-no-pie` does not disable ASLR, so local variables stored on the stack still have different addresses between executions.

# Understand the memory allocation function by following those steps.

**Step 1:** Run this program (without '-no-pie'):
**Step 2:** Copy the output and put it in your report

```
Address of Pointer
>>0x7ffc40e5f770
>>0x7ffc40e5f778

Effective Address
>>0x9700000006
>>(nil)

After malloc Pointer a
>>0x57ca0b518420
>>0x57ca0b518420
>>0x57ca0b518444

Array c
>>0x7ffc40e5f780
>>0x7ffc40e5f780
>>0x7ffc40e5f7a4

After realloc Pointer a
>>0x57ca0b518420
>>0x57ca0b518420
>>0x57ca0b518444
>>0x57ca0b5193bc
```

**Step 3:** Explain why the results show up that way?

# Explanation

At the beginning of the program, a and b are pointer variables stored on the stack. The difference between these addresses is 8 bytes, which is expected because the program is running on a 64-bit system where a pointer normally occupies 8 bytes.

The values of `a` and `b` before they are initialized are not valid values. In the output, b happens to show nil. However, a shows 0x9700000006 because a is an uninitialized local pointer and contains an undefined value.

After `malloc()` allocates memory for 10 integers, the memory is allocated from the heap, often much farther address from what store on a stack. Since an `int` is 4 bytes, the allocated block contains 10 × 4 = 40 bytes. The address returned by `malloc()` is 0x57ca0b518420. Therefore, both `a` and `&a[0]` have the same address:

```
a = 0x57ca0b518420
&a[0] = 0x57ca0b518420
```

The address of `a[9]` is:

```
0x57ca0b518444
```

the diffrent is 24 in hexadecimal and 36 bytes after `a[0]` , because:

```
9 × sizeof(int)
= 9 × 4
= 36 bytes
```

The array `c[10]` is a normal local array, so it is stored on the stack. Its first element has the same address as the array itself, The address of `c[9]` is 36 bytes after `c[0]` , just like `a[9]` .

```
c = 0x7ffc40e5f780
&c[0] = 0x7ffc40e5f780
&c[9] = 0x7ffc40e5f7a4
```

Finally, the program calls realloc()With static, y keeps its value between loop iterations, so the output is 6, 7, and 8. Its address remains the same because all iterations use the same variable similar to step 1.

When using `-no-pie` , PIE is disabled. This means the executable is loaded at a fixed memory address instead of being relocated to a different address between executions. Since the `static` variable y is stored in the executable's data section, its address is also fixed. Therefore, y has the address `0x404018` in every execution.

ASLR is still enabled, but it does not change the address of y because the executable was compiled without PIE. ASLR can still randomize other memory regions, such as the stack and heap.

This requests enough memory for 1000 integers 1000 × 4 = 4000 bytes.

In this particular run, `realloc()` was able to keep the same starting address:

```
0x57ca0b518420
```

The address of `a[999]` is 999 × 4 = 3996 bytes after the beginning of the allocation.

Thus, the output demonstrates that a and c are stored in different memory areas because they are declared differently. The pointer variable a itself is stored on the stack, but after `malloc()` is called, it points to a block of memory allocated on the heap. In contrast, c is a local array declared directly as int c[10];, so the array itself is stored on the stack. Therefore, although both a and c contain 10 integers and have the same size, their actual data is stored in different memory areas. This is why the addresses of a's allocated memory and c are far apart.

**Step 4:** Modify the code by uncomment 2 places, and re-run this program
**Step 5:** Copy the output and put it in your report

```
Address of Pointer
>>0x7ffc5f9c7f40
>>0x7ffc5f9c7f48

Effective Address
>>0x9700000006
>>(nil)

After malloc Pointer a
>>0x5ed29e7fc420
>>0x5ed29e7fc420
>>0x5ed29e7fc444

Array c
>>0x7ffc5f9c7f50
>>0x7ffc5f9c7f50
>>0x7ffc5f9c7f74

After malloc Pointer b
>>0x5ed29e7fc450
>>0x5ed29e7fc450
>>0x5ed29e7fc474

After realloc Pointer a
>>0x5ed29e7fc480
>>0x5ed29e7fc480
>>0x5ed29e7fc4a4
>>0x5ed29e7fd41c
```

**Step 6:** Explain why the result show up that way? And why is the result different from step 2.

**Explanation and Comparison**

# Explanation

After uncommenting the second `malloc()` section, the program also allocates memory for pointer b. Since a `float` normally occupies 4 bytes, this allocates 10 × 4 = 40 bytes

The output shows:

```
After malloc Pointer b

>>0x5ed29e7fc450
>>0x5ed29e7fc450
>>0x5ed29e7fc474
```

The address of `b[9]` is 36 bytes after `b[0]` The important difference occurs when `realloc()` is called for a.

In Step 2, the original allocation for a was followed by `realloc()` , and the allocator was able to extend the allocation without moving it

```
Before realloc: 0x57ca0b518420
After realloc:  0x57ca0b518420
```

However, in Step 5, memory for b was allocated immediately after the memory for a:

```
a: 0x5ed29e7fc420
b: 0x5ed29e7fc450
```

When `realloc()` tried to increase a from 40 bytes to 4000 bytes, there was not enough suitable contiguous space immediately after the existing allocation because another allocated block was occupying that area.

Therefore, `realloc()` moved a to a new location:

```
Before realloc: 0x5ed29e7fc420
After realloc:  0x5ed29e7fc480
```

This explains why the result in Step 5 is different from Step 2. The difference is caused by the different state of the heap. Adding the `malloc()` for b changes the arrangement of dynamically allocated memory, which can cause `realloc()` to move a instead of extending it in place.

| Feature | Step 2 | Step 5 |
| --- | --- | --- |
| `a` initial allocation | `0x57ca0b518420` | `0x5ed29e7fc420` |
| `b` allocation | Not performed | `0x5ed29e7fc450` |
| `a` after `realloc()` | Same address | **Moved to a new address** |
| Reason | Enough adjacent heap space was available | Adjacent space was not suitable because of the additional allocation |
| `a[0]` after realloc | Same as original `a` | Same not the as original `a` |
| `a[999]` | 3996 bytes from start | 3996 bytes from start |